**Blockchain tehnologija i kriptuvalute**

# RAZVOJ DECENTRALIZIRANE TO-DO APLIKACIJE NA SOLANA BLOCKCHAINU

**Lovro Ružman, 2. g. DRB.**

**Osijek 26.01.2026.**

# 1. Uvod i definiranje problema

## 1.1. Problem

Tradicionalne "To-Do" aplikacije (Web2) pohranjuju podatke korisnika na centraliziranim serverima (npr. Google Tasks, Microsoft To-Do). U tom modelu, korisnik nema stvarno vlasništvo nad svojim podacima; oni ovise o pružatelju usluge koji ih može obrisati, cenzurirati ili izgubiti gašenjem servera. Također, privatnost podataka ovisi o sigurnosnim mjerama te centralne točke.

## 1.2. Rješenje

Cilj ovog projekta je izraditi decentraliziranu aplikaciju (dApp) za upravljanje zadacima na Solana blockchainu. U ovom modelu:

1. Vlasništvo: Svaki zadatak je kriptografski povezan s novčanikom (Walletom) korisnika.
2. Otpornost: Podaci žive na blockchainu, ne na jednom serveru.
3. Transparentnost: Logika aplikacije (Smart Contract) je javno vidljiva i nepromjenjiva nakon postavljanja.

# 2. Korišteni alati i tehnologije

Za realizaciju ovog rješenja korišten je moderni "Solana Tech Stack":

1. **Solana Blockchain (Localhost Validator):** Blockchain visokih performansi koji koristi Proof of History (PoH) mehanizam za brze transakcije uz niske naknade. Za potrebe razvoja korišten je lokalni testni validator.
2. **Rust:** Programski jezik niske razine korišten za pisanje pametnog ugovora (Smart Contract). Odabran je zbog sigurnosti memorije i brzine.
3. **Anchor Framework:** Razvojni okvir za Solanu koji pojednostavljuje pisanje pametnih ugovora, rukovanje accountima i sigurnosne provjere (slično kao što React olakšava JavaScript).
4. **Next.js (React):** Frontend okvir za izradu korisničkog sučelja. Omogućuje brzu interakciju s korisnikom.

5. **Solana Web3.js & Wallet Adapter:** Biblioteke koje omogućuju komunikaciju između web stranice (Frontend) i blockchaina (Backend), te upravljanje novčanicima.
6. **Phantom Wallet:** Kripto novčanik (ekstenzija preglednika) koji služi za autentifikaciju korisnika i potpisivanje transakcija.

# 3. Tehnička analiza: Zašto Solana i arhitektura sustava

## 3.1. Usporedba: Solana vs. Ethereum

Pri odabiru blockchain platforme za interaktivnu aplikaciju poput "To-Do liste", ključni faktori su brzina (latencija) i cijena transakcije.

- **Problem Ethereuma**: Ethereum koristi mehanizam koji je siguran, ali spor (cca 15-30 transakcija u sekundi). Zbog velikog opterećenja mreže, naknade (Gas fees) mogu iznositi od nekoliko dolara do nekoliko desetaka dolara po transakciji. Za aplikaciju gdje korisnik samo želi dodati zadatak "Kupi mlijeko", plaćanje 5$ naknade je ekonomski neisplativo i loše korisničko iskustvo.
- **Rješenje Solane**: Solana rješava problem skalabilnosti inovacijom zvanom Proof of History (PoH).
    - Proof of History djeluje kao "kriptografski sat" koji omogućuje čvorovima (validatorima) da se slože oko vremena događaja bez međusobne komunikacije za svaki blok.
    - To omogućuje teoretsku propusnost od 65.000 transakcija u sekundi (TPS).
    - Cijena: Transakcija na Solani košta djelić centa (cca $0.00025).

Zbog potrebe za čestim interakcijama (dodavanje, brisanje, uređivanje zadataka), Solana je jedina logična opcija koja pruža iskustvo slično Web2 aplikacijama (instantno i gotovo besplatno).

## 3.2. Arhitektura: Solana Account Model

Da bismo razumjeli kod, moramo razumjeti specifičnu arhitekturu Solane koja se razlikuje od ostalih blockchaina. Solana koristi Account Model gdje je logika strogo odvojena od podataka.

1. Program (Stateless): Naš Smart Contract (napisan u Rustu) je samo "izvršni kod". On ne pamti podatke unutar sebe. On je kao funkcija koja prima parametre, obrađuje ih i sprema rezultat negdje drugdje.
2. Data Accounts (Stateful): Svaki zadatak koji kreiramo je zapravo novi, zasebni "Account" (račun) na blockchainu koji pohranjuje podatke (tekst zadatka, status, vlasnika).
3. Rent: Budući da pohrana podataka na blockchainu troši resurse validatora, svaki Account mora platiti malu količinu SOL-a (Rent) da bi ostao živ.

## 3.3. Analiza Backenda (Rust & Anchor)

Backend je napisan koristeći Anchor Framework, koji apstrahira složene sigurnosne provjere Solane.

Struktura podataka (Data Structure): Prvo definiramo kako jedan zadatak izgleda na blockchainu. Ovo je "nacrt" za naše Data Accounte.

```
#[account]
pub struct Task {
  pub author: Pubkey,     // Tko je vlasnik (32 bajta)
  pub is_done: bool,      // Status zadatka
  pub description: String, // Tekst zadatka
  pub created_at: i64,    // Vrijeme kreiranja
}
```

**Logika:** Glavna funkcija add_task pokazuje kako se podaci inicijaliziraju. Ovdje koristimo Context da pristupimo accountima.

```
pub fn add_task(ctx: Context<AddTask>, description: String) -> Result<()> {
  let task = &mut ctx.accounts.task;
  let author = &ctx.accounts.author;
  // Validacija: Tekst ne smije biti predug
  if description.chars().count() > 400 {
    return Err(ErrorCode::TextTooLong.into());
  }
  // Postavljanje vrijednosti
  task.author = *author.key;
  task.is_done = false;
  task.description = description;
  Ok(())
}
```

Ovaj kod demonstrira sigurnost: Anchor automatski provjerava je li onaj tko šalje transakciju (Signer) stvarno vlasnik računa koji plaća naknadu.

## 3.4. Analiza Frontenda (Integracija)

Frontend aplikacija (React) ne posjeduje bazu podataka. Svaki put kad se stranica učita, ona "pita" blockchain za podatke.

Povezivanje s Blockchainom: Koristimo biblioteku @solana/web3.js za komunikaciju. Ključni dio je funkcija koja šalje transakciju.

```
const addTask = async () => {
  // Generiramo novu adresu za zadatak
  const taskKeypair = web3.Keypair.generate();
  // Šaljemo RPC poziv na blockchain
  await program.methods
   .addTask(inputText)
   .accounts({
    task: taskKeypair.publicKey,
    author: wallet.publicKey,
    systemProgram: web3.SystemProgram.programId,
   })
   .signers([taskKeypair]) // Potpisujemo kreiranje novog računa
   .rpc();
};
```

Ovdje je vidljivo da Frontend mora generirati novi par ključeva (Keypair) za svaki novi zadatak, jer kao što smo objasnili u arhitekturi – svaki zadatak je svoj vlastiti račun na mreži.

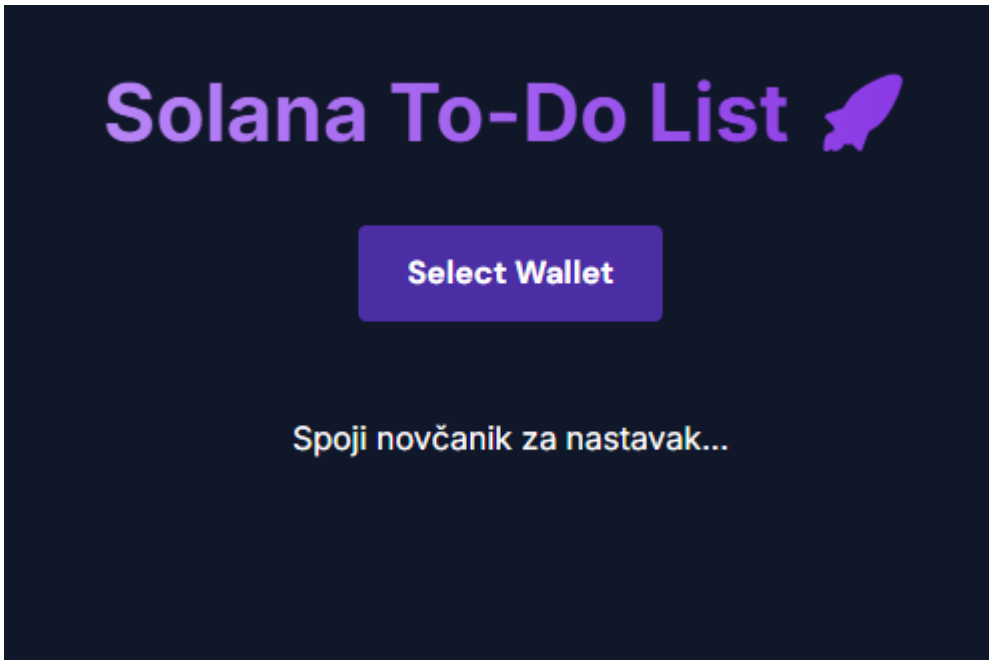
# 4. Prikaz rada aplikacije

U ovom poglavlju demonstriran je rad aplikacije kroz ključne korake: od povezivanja novčanika do upravljanja zadacima.

## 4.1. Početno stanje i povezivanje

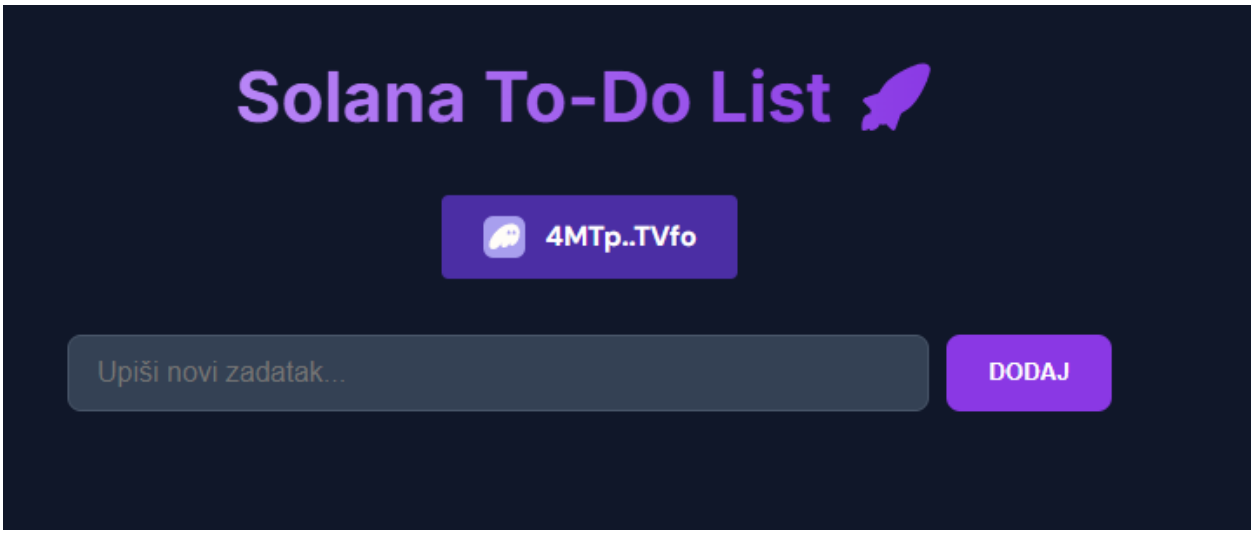
Prilikom pokretanja aplikacije, korisniku se prikazuje minimalističko sučelje. Budući da se radi o Web3 aplikaciji, nema klasične registracije e-mailom. Umjesto toga, koristi se kripto-novčanik za identifikaciju.

Na Slici 1 prikazan je početni zaslon na kojem je vidljiv gumb za odabir novčanika ("Select Wallet").




*Slika 1: Početno sučelje aplikacije prije povezivanja korisnika.*

Nakon odabira Phantom novčanika i uspješnog povezivanja, aplikacija dohvaća javni ključ korisnika i prikazuje ga u ljubičastom okviru, što potvrđuje da je sesija aktivna (Slika 2).

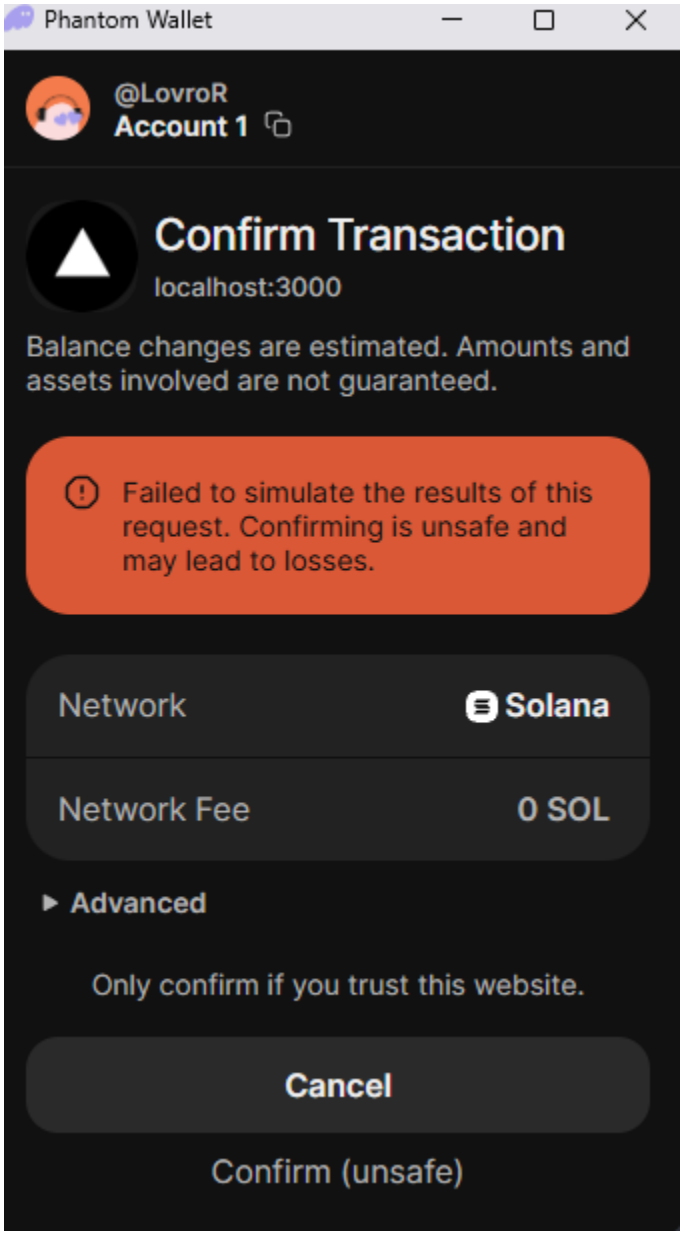



*Slika 2: Uspješno povezivanje wallet-a.*

## 4.2. Proces kreiranja zadatka (On-Chain transakcija)

Korisnik unosi tekst zadatka (npr. "Kupi namirnice") i inicira spremanje klikom na gumb "DODAJ". Za razliku od običnih web aplikacija, ovdje se podatak ne sprema tiho u pozadini.

Kako je prikazano na Slici 3, Phantom novčanik otvara skočni prozor (pop-up) koji od korisnika traži digitalni potpis i odobrenje troška transakcije (Network Fee). Ovo je ključni sigurnosni mehanizam Solane.



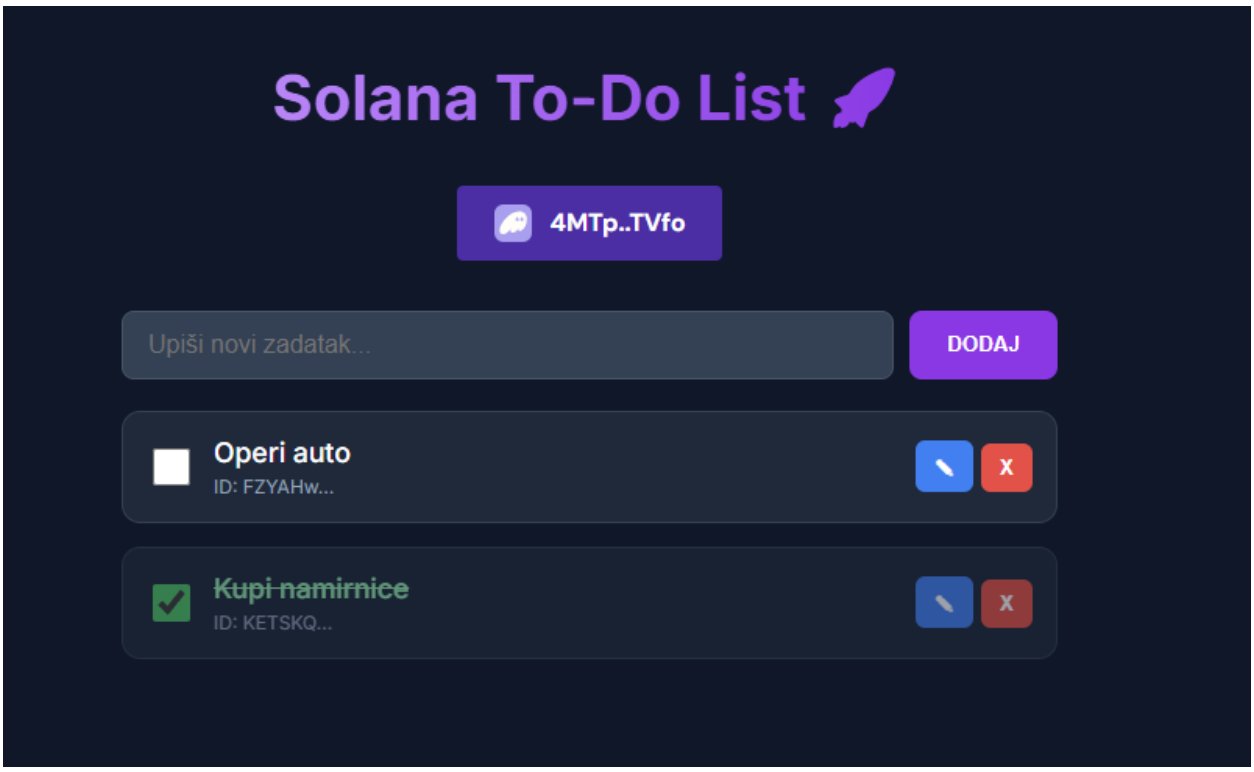

*Slika 3: Zahtjev za potpisivanje transakcije unutar Phantom novčanika..*

## 4.3. Upravljanje listom zadataka

Nakon što je transakcija potvrđena na blockchainu (što na Solani traje svega nekoliko sekundi), zadatak se pojavljuje na listi. Svaki zadatak ima svoj jedinstveni ID (adresu računa).

Konačni izgled aplikacije s nekoliko zadataka vidljiv je na Slici 4. Ovdje možemo vidjeti:

1. Zeleni (prekriženi) zadatak: Prikazuje vizualnu potvrdu da je status is_done na blockchainu promijenjen u true.
2. Opcije upravljanja: Desno od svakog zadatka nalaze se gumbi za uređivanje (olovka) i brisanje (X).




*Slika 4: Prikaz liste zadataka s funkcionalnostima uređivanja i brisanja.*

# 5. Zaključak

U ovom seminarskom radu uspješno je projektirana i implementirana decentralizirana aplikacija (dApp) za upravljanje zadacima na Solana blockchainu. Kroz praktičan rad demonstrirana je sinergija modernih Web3 tehnologija: Rust i Anchor Framework za backend logiku, te Next.js i Solana Web3.js za korisničko sučelje.

Ključni ishodi ovog projekta su:

1. Razumijevanje arhitekture: Usvojen je koncept Account Modela na Solani, gdje se logika programa strogo odvaja od pohrane podataka, što se bitno razlikuje od tradicionalnog razvoja softvera.

2. Funkcionalnost: Aplikacija podržava cjelokupni CRUD (Create, Read, Update, Delete) ciklus. Svaka operacija rezultira stvarnom transakcijom na mreži, osiguravajući nepromjenjivost i transparentnost podataka.

3. Korisničko iskustvo: Iako blockchain aplikacije nose određenu kompleksnost (potreba za novčanikom, potpisivanje transakcija), korištenjem brzog blockchaina poput Solane postignuto je iskustvo korištenja koje je kompetitivno klasičnim web aplikacijama.

Ovaj projekt poslužio je kao dokaz koncepta da je moguće izgraditi sigurne i brze aplikacije gdje korisnici imaju apsolutno vlasništvo nad svojim podacima, bez oslanjanja na centralizirane servere. Budući razvoj mogao bi uključivati dodavanje naprednijih funkcionalnosti poput dijeljenja zadataka s drugim novčanicima ili integraciju notifikacija.

# 6. Literatura

Solana Labs. (2024). Solana Documentation. Dostupno na: https://docs.solana.com [Pristupljeno: 26.01.2026.]

Coral. (2024). The Anchor Framework Book. Dostupno na: https://www.anchor-lang.com [Pristupljeno: 26.01.2026.]

Solana Foundation. (2024). Solana Cookbook. Dostupno na: https://solanacookbook.com [Pristupljeno: 26.01.2026.]

Klabnik, S. & Nichols, C. (2023). The Rust Programming Language. San Francisco: No Starch Press. Dostupno na: https://doc.rust-lang.org/book [Pristupljeno: 26.01.2026.]

Phantom Technologies. (2024). Phantom Wallet Developer Docs. Dostupno na: https://docs.phantom.app [Pristupljeno: 26.01.2026.]